

Part II

Bridges From Reinforcement Learning to Causal Inference



Tabular Reinforcement Learning

“Most days I did not have lunch. As a matter of fact, I very clearly remember that my chief fantasy on the subway train going home was that we would be marooned and have to turn cannibal. I would look at my fellow passengers and wonder who I would eat first.”

Richard Bellman (Inventor of Dynamic Programming)

At the intersection of decision-making and optimization, Reinforcement Learning has revolutionized the modern technical conceptualization of sequential decision problems. Reinforcement Learning addresses a question that Causal Inference alone cannot fully answer: How should we act when our decisions today affect not only immediate outcomes but also future states and possibilities? Typically Causal Inference will avoid this question, deferring it to the researcher applying the methods.

Reinforcement Learning, in contrast, focuses directly on determination of optimal actions, considering “why” models only as a means to the end of decision-making.

4.1 Multi-Arm Bandit Problems

Let us return to the clinical trial situation from earlier, where we discussed various adaptive designs such as “play-the-winner”. Another adaptive clinical trial design involves reframing the problem in Reinforcement Learning terms and using its associated methods to optimize the outcome.

In Reinforcement Learning, a *k*-armed Bandit Problem is any problem in which one is repeatedly faced with a choice among *k* different options, with the objective of maximizing some numerical reward drawn from a fixed distribution. Each action-selection instance is called a “trial”. In this book, trials are always indexed using *j*. The treatment assignment problem discussed above can be framed as a *k*-armed bandit problem: Here, the numerical reward is the tumor volume reduction following each patient’s treatment, and the *k* different options are different drugs that the doctor could prescribe (*k* = 2 in the above example). Each trial corresponds to a new patient. Figure 4.1 heuristically shows the idea of a 3-armed clinical bandit; the small oracular robot face is the decision-making agent that prescribes treatments according to some internal policy.

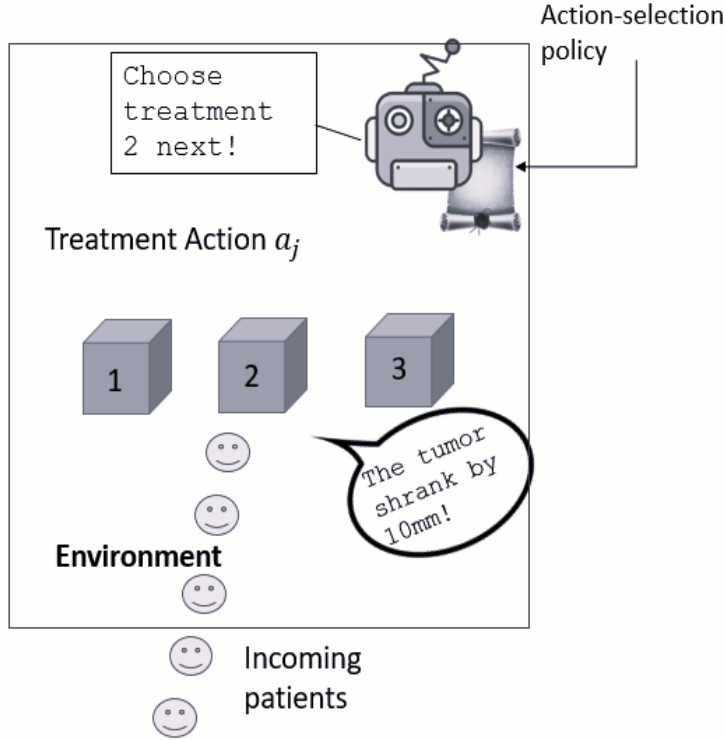


FIGURE 4.1: Heuristic showing a 3-armed clinical bandit problem.

In Reinforcement Learning, trials are often referred to as “timesteps”. I will not use this terminology in this book because it would be very confusing when we extend our example to the case where each patient receives multiple sequential treatments at different timepoints.

Compared to the causal approaches in earlier chapters, the formulation of the clinical trial as a k -armed bandit problem encodes a subtle shift in the teleology - i.e. the *goal*, final end or purpose - of the experiment. Instead of asking “what effects does this treatment cause?”, now we are seeking to answer a different question: “how can we modify our actions to achieve the optimal outcome?”.

4.2 Action-Value Estimation

Each treatment arm k has an associated expected population-level tumor reduction θ_k . If we knew the values of θ_k , then maximizing the expected long-term reward would be trivial: One would simply choose the option (machine or path or treatment) which had the highest value (probability of reward) every time, setting $\pi(a_j = a^{(k)}) = 1$ if $k = \arg \max_k \theta_k$ and zero otherwise. But in practice we do not know the θ_k . A simple Reinforcement Learning

approach to this estimation problem is the *action-value* method.

In lieu of the true values of θ_k , action-value methods typically involve generating estimates $\hat{\theta}_k$ of θ_k which are updated after each trial, i.e. following the observation of each treated patient's outcome. If we estimate by computing the average of the rewards for each treatment arm, then these estimates can be computed efficiently by recursively updating the previous estimate instead of recomputing the average. To see this, consider the formal definition of the average of the observed rewards for action $a^{(k)}$ following trial j (Equation 4.1), where r_j is the tumor reduction observed from patient j . In this section we assume one observation per subject, so trial-number and subject-number coincide; the single index j counts both.

$$\bar{r}_j^{(k)} = \frac{\sum_{\ell=1}^j r_\ell \mathbb{1}(a_\ell = a^{(k)})}{\sum_{\ell=1}^j \mathbb{1}(a_\ell = a^{(k)})} \quad (4.1)$$

For ease of notation, let us suppress all of the j terms where $a_j \neq a^{(k)}$ (so we are only trying to compute the effectiveness of a single arm, therefore excluding data which were generated by selecting other arms). Then the average can be rewritten as follows in Equation 4.2, where the final form shows that the new average equals the old average plus a correction term proportional to the “surprise” ($r_j - \bar{r}_{j-1}$), weighted by the learning rate $1/j$.

$$\begin{aligned} \bar{r}_j &= \frac{1}{j} \sum_{\ell=1}^j r_\ell \\ &= \frac{1}{j} \left(r_j + \sum_{\ell=1}^{j-1} r_\ell \right) \\ &= \frac{1}{j} (r_j + (j-1)\bar{r}_{j-1}) \\ &= \bar{r}_{j-1} + \frac{1}{j} (r_j - \bar{r}_{j-1}) \end{aligned} \quad (4.2)$$

Here ℓ is a dummy summation index ranging over trials from 1 to j .

The recursive estimation principle in Equation 4.2 is the foundational premise for Reinforcement Learning, as well as for many popular methods of extracting signal from noise in cases where the dataset is very large. For example, a simple electrical circuit with a resistor and a capacitor performs the continuous-time analogue of this computation: the capacitor's voltage maintains a running (exponentially weighted) average of the incoming signal, smoothing out noise from datapoints that may be accrued every millisecond.

Following each trial, there exists an action that has the highest expected reward: $a^{(k^*)} = \arg \max_k \hat{\theta}_k$. In Reinforcement Learning this $a^{(k^*)}$ is known as the *greedy* action. Similarly, the action-selection policy π which returns $a^{(k^*)}$ at every trial is called the greedy policy. Usually, Reinforcement Learning literature suppresses the action index k and simply uses: $a^* = \arg \max_a \hat{\theta}_a$; we will follow this convention moving forward.

The greedy policy immediately leverages all of the existing data to gain the maximum reward on subsequent trials: In Reinforcement Learning we say that the greedy action *exploits* existing information. However, it can converge to suboptimal results because it lacks an early sampling period where actions are chosen randomly, which is often necessary to avoid getting stuck at a local optimum. Reinforcement Learning convention calls this early sampling period the *exploration* phase. Note, however, the more we explore by selecting actions randomly, the greater chance we have at randomly selecting bad actions. As soon as we are confident in our estimate of the action-value function, then, we should stop exploring and exploit existing knowledge. This is the same problem as described in the discussion of

sample size calculations, where as soon as we are confident that one drug is better than the other, we must stop the RCT and give all patients the better drug.

To accommodate both exploration and exploitation, Reinforcement Learning methods often employ a policy which generally selects the greedy action, but with some probability ϵ could return a random action. Typically ϵ will start at 1 and decrease as the trials progress. The conventional statistical approach to an RCT can be viewed as a special case of an ϵ -greedy policy, where $\epsilon = 1$ until clinical efficacy is established, at which point ϵ decreases abruptly to zero. The ϵ -greedy strategy illustrated in Figure 4.2 ensures sufficient exploration in early trials while gradually transitioning to exploitation as confidence in the estimates increases.

Definition 4.1 (ϵ -greedy) A policy π is ϵ -greedy if and only if it returns a uniformly random action with probability ϵ , and the greedy action $\arg \max_a \hat{\theta}_a$ with probability $1 - \epsilon$.

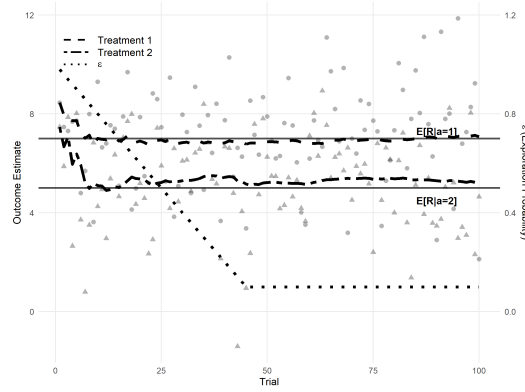


FIGURE 4.2: Exploration-exploitation trade-off in an ϵ -greedy policy. The figure shows how the outcome estimates for two treatments converge as trials progress. Initially with high exploration ($\epsilon = 1$), both treatments are sampled equally. As the algorithm learns that Treatment 1 performs better, it reduces exploration and increasingly selects Treatment 1, demonstrating the transition from exploration to exploitation.

The Q-value estimates plotted in Figure 4.2 are running averages of observed rewards; in R, for a single arm's reward vector, this is `Q <- cumsum(rewards) / seq_along(rewards)`.

Both i) the sample size calculations discussed in section 2.2 and ii) methods to balance the exploration/exploitation tradeoff encode solutions to the problem that in order to know that action 1 is better than action 2, we have to try action 2 (the bad action) a few times. Furthermore, what number of times we need to try the bad action to be sure it's bad depends on the variability between trials of the same action. Solutions to this problem often perform some max-min optimization where we try to maximize our information gained from the sequence of trials while simultaneously minimizing the number of bad actions chosen.

Consider, for example, a clinical trial comparing two different treatment arms for asthma: Treatment $a^{(1)}$, the standard-of-care drug, and treatment $a^{(2)}$, a novel candidate. Once again we will use the asthma exacerbation rate reduction as an outcome measure (note that we use the *reduction* from baseline, so higher is better).

In conventional RCTs, patients would be randomized with equal probability between both treatments for the entire duration of the study. However, in an adaptive trial using action-value methods, we might begin with an exploration phase where $\epsilon = 1$, randomizing the first 30 patients equally between treatments.

After these initial 30 trials, we compute the action-value estimates in Equations 4.3 and 4.4. In this example, patients assigned to treatment 2 (1) had an average of 3.7 (2.3) fewer exacerbations following treatment than they did before treatment.

$$\hat{\theta}_1 = 2.3 \quad (4.3)$$

$$\hat{\theta}_2 = 3.7 \quad (4.4)$$

Using the recursive estimation principle from Equation 4.2, we continually update our estimate of $\hat{\theta}_k$ after observing each subsequent patient outcome. Under an ϵ -greedy policy with $\epsilon = 0.3$ and two treatment arms, action selection proceeds as follows: With probability 0.3, select uniformly at random between both treatments; with probability 0.7, select the treatment with the higher estimated value. Equation 4.5 follows, where $0.15 = 0.3/2$ is the probability of randomly selecting each action.

$$\pi(a_j = a^{(k)}) = \begin{cases} 0.7 + 0.15 = 0.85, & \text{if } k = \arg \max_{k'} \hat{\theta}_{k'} \\ 0.15, & \text{otherwise} \end{cases} \quad (4.5)$$

Consequently, 85% of subsequent patients receive the apparently superior treatment 2 ($a^{(2)}$): 70% through greedy selection, plus half of the 30% who continue to be randomly assigned to treatments. This ϵ -greedy approach simultaneously addresses the need to provide what appears to be the better treatment to more patients while also maintaining sufficient exploration to verify our initial findings.

After an additional 40 patients, perhaps the estimates converge to $\hat{\theta}_1 = 2.1$, $\hat{\theta}_2 = 4.2$. As confidence in our estimates increases, we might further decrease ϵ to 0.1, assigning 95% of new patients to the more effective treatment while continuing minimal exploration. Once we are sufficiently convinced that $\theta_2 > \theta_1$, we terminate the trial and assign all patients drug 2.

The Jiu Jitsu submission selection problem discussed in previous chapters can also be framed as a multi-armed bandit problem. Suppose you are a beginner BJJ practitioner, and your arsenal includes $k = 5$ different submission attempts (Table 4.1).

Submission	Action
Triangle choke	$a^{(1)}$
Rear naked choke	$a^{(2)}$
Armbar	$a^{(3)}$
Kimura	$a^{(4)}$
Guillotine	$a^{(5)}$

TABLE 4.1: Action space for BJJ submission selection.

Each rolling session represents a trial j , and the reward r_j could be binary (1 for successful submission, 0 for escape) or continuous (e.g., time to submission, with faster submissions receiving higher rewards)¹. Each submission technique has an associated true success rate θ_k that reflects your personal proficiency with that technique against opponents of your skill level.

The exploration-exploitation principle applies to this scenario as well: If you always attempt the triangle choke because it worked well in your first few attempts, you might

¹Unlike clinical trials where patients are independent, rolling sessions exhibit temporal dependence as skill improves. We simplify by treating sessions as independent trials, though more sophisticated models could account for learning curves.

miss discovering that you're actually much more effective with the Kimura. Conversely, if you spend too much time exploring less effective submissions, you will miss opportunities to secure victories with your best techniques.

An ϵ -greedy approach might work as follows: For the first 100 rolling sessions, you explore uniformly ($\epsilon = 1$), attempting each submission type roughly equally to gather initial data. After computing the preliminary success rates in Table 4.2, you might then transition to an ϵ -greedy policy with $\epsilon = 0.3$: 76% of the time you attempt your best-performing submission (Kimura) (70% through greedy selection, plus $0.3/5 = 6\%$ through random exploration), while the remaining 24% of the time you continue exploring other techniques.

Submission	Parameter	Success Rate
Triangle	$\hat{\theta}_1$	0.65
Rear naked choke	$\hat{\theta}_2$	0.45
Armbar	$\hat{\theta}_3$	0.70
Kimura	$\hat{\theta}_4$	0.80
Guillotine	$\hat{\theta}_5$	0.35

TABLE 4.2: Preliminary success rate estimates after initial exploration phase.

Using the recursive estimation principle from Equation 4.2, you update your success rate estimates after each rolling session per Equation 4.6, where n_k is the number of times you've attempted submission k .

$$\hat{\theta}_k^{(j+1)} = \hat{\theta}_k^{(j)} + \frac{1}{n_k + 1} (r_{j+1} - \hat{\theta}_k^{(j)}) \quad (4.6)$$

ϵ -greedy sampling is still commonly used in practical RL applications due to its ease of implementation, low computational overhead, and general effectiveness. However, in cases where samples are limited or the cost of exploration is extremely high (such as in clinical trials), a more nuanced approach may be necessary. Bayesian methods for adaptive sample selection are popular for these applications; for a detailed discussion on adaptive sampling methods (such as Thompson sampling) see [142]. For a discussion of applications of these methods to clinical trials, see [176].

4.3 Markov Decision Processes

Most real-world problems are more sophisticated than the bandit problem. These typically require some additional mathematical machinery; usually one would begin an RL application by formulating the problem as a stochastic decision process (SDP). A stochastic decision process is a collection of states, actions, and subsequent rewards observed from a dynamic system progressing over time. In the clinical example, states might include subject ID, patient covariates, and disease severity, actions might correspond to treatment decisions, and the reward could be some measure of patient wellbeing. A formal definition of a stochastic decision process follows.

Definition 4.2 (Stochastic Decision Process (SDP)) *A Stochastic Decision Process is a 4-tuple $(\Omega, \mathcal{A}, \Gamma, \rho)$ where:*

- Ω is the **state space**: the set of all possible states the system can be in. States are denoted X (when treated as a random variable) or x (when treated as a specific value).
- $\mathcal{A}$ is the **action space**: the set of actions available to the agent. Actions are chosen according to a policy $\pi(a | x)$, which gives the probability of taking action a when the system is in state x .
- Γ is the **transition rule**: $\Gamma(x' | x, a)$ is the conditional probability of moving to state x' given current state x and action a .
- ρ is the **reward function**: $\rho : \Omega \rightarrow \mathbb{R}$, where $\rho(x')$ is the reward received upon arriving at state x' .

The system evolves over indexed time steps j :

1. Initialize x_0 ; set $j = 0$.
2. Draw $a_j \sim \pi(\cdot | x_j)$.
3. Apply a_j ; draw $x_{j+1} \sim \Gamma(\cdot | x_j, a_j)$.
4. Compute $r_{j+1} = \rho(x_{j+1})$.
5. Set $j = j + 1$ and return to step 2.

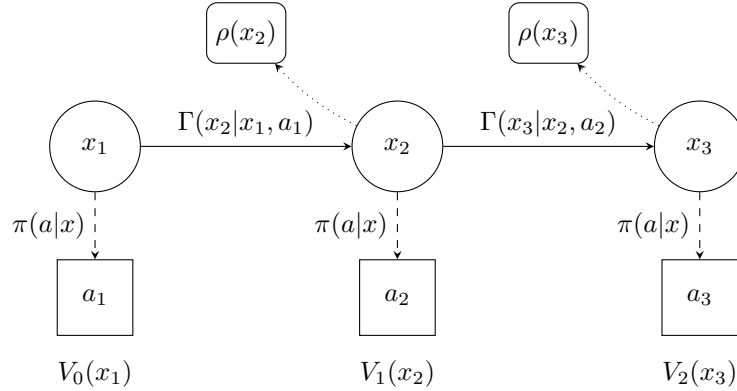


FIGURE 4.3: Illustration of the standard MDP framework. States x are fully observable, and the agent selects actions a according to policy π . State transitions follow the dynamics Γ , and rewards ρ are determined by the resulting state.

Conventional Reinforcement Learning notation does not differentiate R and ρ , using lowercase r interchangeably as a random variable, a fixed value, or a function. This dual usage of r as a variable and a function can be very confusing, so I use the more precise notation here: ρ is a function, computed using the state x' . The item that ρ outputs, before the state transition, is a random variable, denoted R . Once the state transition is performed and the reward has been computed, the result is a *value*, i.e. a specific random draw of R , which we denote r .

If a stochastic decision process satisfies the Markov property (Definition 4.3), then it is called a *Markov Decision Process* (MDP). A version without the action $\mathcal{A}$ is simply called

a *Markov process* (or *stochastic process* if the Markov property does not hold). There is a continuous version of a stochastic process as well, where $j \in \mathbb{R}$ instead of $j \in \mathbb{Z}$; the canonical example is the *Wiener process* (also known as Brownian motion), a continuous-time process with independent Gaussian increments, which is used in physics to describe various forms of diffusion, among other things. Models driven by Wiener processes have also been developed in financial forecasting, where they are used to model the behavior of the stock market. One example is the *Stochastic Alpha-Beta Rho* (SABR) model, which makes predictions based on the underlying asset price w_1 and its variability w_2 , where the dynamics of w_1, w_2 are driven by Wiener processes². The monograph “Modern SABR analytics: formulas and insights for quants, former physicists and mathematicians” [3] provides an excellent overview of this technique.

Definition 4.3 (Markov Property) *A Stochastic Decision Process $(\Omega, \mathcal{A}, \Gamma, \rho)$ is said to satisfy the Markov Property if and only if for any time step $j \geq 0$ and for all states $x_{j+1} \in \Omega$, the conditional probability distribution of the next state depends only on the current state and action, and is independent of the history of previous states and actions. Formally, for any sequence of states and actions $\{x_0, a_0, x_1, a_1, \dots, x_j, a_j\}$:*

$$\Gamma(x_{j+1}|x_j, a_j, x_{j-1}, a_{j-1}, \dots, x_0, a_0) = \Gamma(x_{j+1}|x_j, a_j) \quad (4.7)$$

Gridworld is the classic sandbox example that Reinforcement Learning researchers typically use to teach Q -learning, or to experiment with new methods on [164]. The grid world environment consists of an $n \times n$ grid where each square contains a specific reward. The agent can occupy one square at a time, and has 4 possible actions, corresponding to moving left, right, up, or down. Typically there is one “target” square with a high value; if the agent reaches this square then the episode ends and the game resets. Often there are also one or more “trap” squares that the agent must avoid; if the agent lands on a trap, then the episode ends with a large penalty. The agent usually receives either no reward or a small penalty for landing on all other squares, in order to incentivize efficient progress towards the target. Figure 4.4 shows a visualization of the Q function in a 5×5 grid world environment where the goal square is on the top right corner, and the trap is on the bottom left. In grid world, the state is the tuple (x, y) consisting of the x and y coordinates of the agent’s current position, and the action space is $\mathcal{A} = \{\text{left, right, up, down}\}$ as previously indicated.

A Python implementation of a grid world environment follows.

²An “asset” could be a stock, or a combination of stocks, such as an index fund.

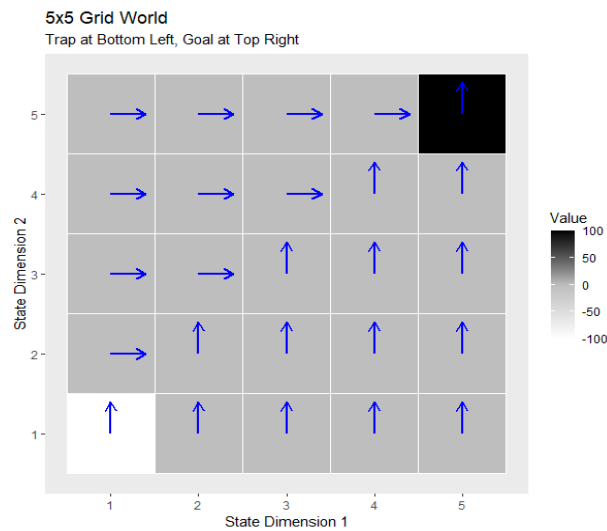


FIGURE 4.4: Visualization of a 5x5 grid world. Arrows indicate potential actions that the agent might take.

Grid World Environment Class

```
# gridworld.py

# Environment class
class GridWorldEnv:
    def __init__(self, grid_size=5):
        self.actions= ['right', 'down', 'left', 'up']
        self.action_to_delta = {'right': (0, 1), 'down': (1, 0),
                                'left': (0, -1), 'up': (-1, 0)}
        self.action_space = len(self.actions)
        self.grid_size = grid_size
        self.obs_space = 2 # x, y position

    def reset(self):
        self.pos = (0, 0)
        self.history = [self.pos]
        return np.array(self.pos)

    def step(self, action_idx):
        action = self.actions[action_idx]
        dx, dy = self.action_to_delta[action]

        new_x = np.clip(self.pos[0] + dx, 0, self.grid_size - 1)
        new_y = np.clip(self.pos[1] + dy, 0, self.grid_size - 1)
        self.pos = (new_x, new_y)
        self.history.append(self.pos)

        reward = -0.1
        done = False
```

```

        if self.pos == (0, self.grid_size - 1): # Goal (top right)
            reward = 10
            done = True
        elif self.pos == (self.grid_size - 1, 0): # Trap (bottom left)
            reward = -10
            done = True

    return np.array(self.pos), reward, done

```

Recall from Section 1.4 that one difference between Python and R programming is that R is mostly a *functional* language, while Python can also lend itself well to *object-oriented* strategies. The above example is a simple example of object-oriented programming (OOP): Instead of accomplishing our task through a sequence of independent functions, as we have done in previous examples, we define an object *class* with various attributes, e.g. `self.grid_size`. An example of object usage follows; here, we implement a simple interactive visualization of grid world, assuming that the grid world object has another function as an attribute labeled “render” which visualizes the function (the “render” function is rather long and tedious, so we omit printing it here).

Grid World Interaction Wrapper

```

import pandas as pd # data processing, CSV file I/O
import matplotlib.pyplot as plt # plotting
import matplotlib.patches as patches # plotting
from IPython.display import clear_output
import time

# Interactive wrapper
class GridWorldWrapper:
    def __init__(self, env):
        self.env = env
        self.total_reward = 0

```

Within each class, the function `__init__()` is automatically run upon object initialization. Function names which both begin and end with two underscores, such as `__init__`, are called “dunder” (double underscore) methods: names that Python reserves for special built-in behaviors, such as initialization. Relatedly, attributes whose names begin (but do not end) with two underscores are *name-mangled*: Python internally renames them by prepending the class name, which protects them from being accidentally overwritten in subclasses. With these classes defined, we can execute the interactive environment as follows; this implementation allows the user to manually navigate the grid world. The `GridWorldWrapper` class has the following functions:

```
def play_interactive(self):
    # Turn on interactive mode
    plt.ion()

    state = self.env.reset()
    self.total_reward = 0
    done = False
    step_count = 0

    # Create figure once
    fig, ax = plt.subplots(1, 1, figsize=(7, 7))
```

In this first function `play_interactive`, we initialize the graphics and then enter a display loop that updates and visualizes the state based on user inputs.

```
while not done:
    # Update display
    self.env.render(ax=ax)
    plt.draw()
    plt.pause(0.01) # Small pause to update display

    action_str = input(f"\n Position {self.env.pos} |
    Step {step_count + 1} |
    Total Reward: {self.total_reward:.2f}\n
    Enter action (0=Right, 1=Down, 2=Left, 3=Up) or
    'q' to quit: ")

    if action_str.lower() == 'q':
        print("Quitting game.")
        plt.close()
        break

    try:
        action = int(action_str)
        if action not in range(4):
            print(" Invalid action! Use 0-3")
            continue
    except ValueError:
        print(" Invalid input! Enter a number 0-3")
        continue

    state, reward, done = self.env.step(action)
    self.total_reward += reward
    step_count += 1

    print(f"Action: {self.env.actions[action]} ->
    New Position: {state}, Reward: {reward:.2f}")
```

```

# Final render
self.env.render(ax=ax)
plt.draw()
plt.pause(0.01)

print(f"\n{'='*40}")
if self.total_reward > 0:
    print("SUCCESS! You reached the goal!")
else:
    print("FAILED! You hit the trap!")
print(f"Total reward: {self.total_reward:.2f}")
print(f"Steps taken: {step_count}")
print(f"{'='*40}")

# Keep window open
plt.ioff()
plt.show()

```

After creating the environment class object, we then need to actually call an instance of the object in order to run the system. In Python projects, typically one would have one module (Python script) containing the class definition, and then a secondary execution script, commonly titled “main” or similar, that imports the class and runs the code. For smaller projects, it may be feasible to create the entire pipeline in a single script. However, for most practical projects, it is convenient to separate these modules early on so that the codebase stays organized and readable as it grows.

```

# Create environment
env = GridWorldEnv(grid_size=5)
wrapper = GridWorldWrapper(env)

wrapper.play_interactive()

```

Notice how this Python code differs from previous strategies in R: While R mainly functions “horizontally”, linearizing processes into a sequence of iterative steps, Python programming is more “vertical”; we parallelize by adding dimensions and features to a fixed set of objects to solve the problem. Loosely speaking, Python generally organizes code by *stacking*, while R tends to organize code by *flattening*.

For a more realistic example, let us return to the clinical treatment scenario. Suppose for a moment that we are trying to minimize the eventual tumor volume from each patient, where the tumor is measured before treatment and again at some specific day. We additionally record patient-specific covariates such as current age, smoking status, and age at onset (for the purposes of illustration, assume those are the only covariates measured). This problem can be formulated as an MDP in two equivalent ways that differ only in their indexing scheme (Figure 4.5).

In the first scenario, the MDP is a collection of two-state episodes, where each two-state episode corresponds to the before and after states from a single subject. Here, the state X_j corresponding to the t^{th} observation of patient i would be a vector $(x[1], x[2], x[3], x[4], x[5])$ containing the patient’s current age, smoking status, age of onset, and tumor volume respectively, plus a fifth element $x[5] = t$ recording the within-patient timepoint ($t = 0$ pre-treatment, $t = 1$ post-treatment). Some variables (such as age of onset) may be con-

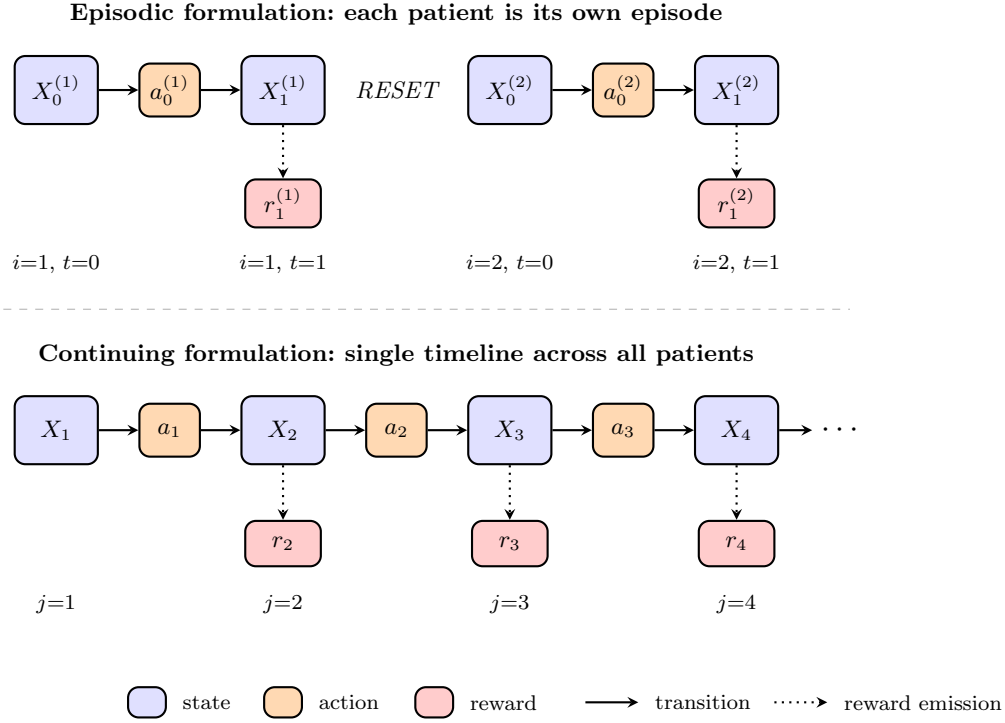


FIGURE 4.5: Episodic versus continuing MDP formulations for the sequential clinical trial problem. **Top:** Episodic formulation. Each patient is treated as an independent episode with states indexed by patient i and within-patient timepoint t ; episodes reset between patients. **Bottom:** Continuing formulation. Patients are concatenated into a single timeline indexed by j , with each state including the previous patient's final tumor volume to define the reward, though this does not affect state transitions.

stant across both observations of each patient. Others (such as tumor volume) might vary. Some states might include missing values: This would be the case, for example, if the size of the tumor was only measured post-treatment. The reward $\rho(X) : \Omega \rightarrow \mathbb{R}$ might be defined as a function which returns the negative of tumor volume from the post-treatment states, as in Equation 4.8.

$$\rho : \Omega \rightarrow \mathbb{R} | \rho(X) = \begin{cases} -x[4] & x[5] = 1 \\ 0 & \text{otherwise} \end{cases} \quad (4.8)$$

Here, the state space Ω is $\mathbb{N} \times \{\text{Smoker, Non-smoker}\} \times \mathbb{N} \times \mathbb{R} \times \{0, 1\}$.

The second approach formulates the problem as a continuing MDP with each trial representing a new subject. Each state x_j is a vector containing 5 elements:

1. $x_j[1]$ Current age of patient j
2. $x_j[2]$ Smoking status of patient j
3. $x_j[3]$ Age of onset from patient j
4. $x_j[4]$ Baseline tumor volume of patient j
5. $x_j[5]$ Final tumor volume of patient $j - 1$

The reward function $\rho(x')$ would be defined so that $\rho(x') = -x'[5]$, returning the negative of the final tumor volume from the previous patient. The state transition mechanics Γ

would likely dictate that $\Gamma(x_{j+1}|a_j, x_j) = \Gamma(x_{j+1}|a_j, x_j[1], x_j[2], x_j[3], x_j[4])$, so that the final tumor volume is a function both of the action a_j (the treatment applied), as well as the patient’s covariate vector (current age, age at onset, smoking status, and baseline tumor volume), but **not** of the tumor volume of the previous patient. This second formulation is much less intuitive, but it can be helpful to conceptualize for efficient programming: In the wide formulation, one may frequently have to loop over two indices, subject i and timepoint $t \in \{0, 1\}$. The long format only requires one index, j . However, if there were multiple timepoint observations per subject, then one would use an episodic MDP formulation of the problem; a continuing version would ignore the correlation between observations collected from the same subject.

Wide and Long Formats in Database Design

The wide-long duality also appears in database design. Relational databases (SQL) naturally support long format through normalized table design: a `patients` table and a separate `visits` table linked by `patient_id`. The relational database avoids redundancy and gracefully handles variable numbers of observations per subject. In contrast, document databases (e.g., MongoDB) often store denormalized data resembling wide format, embedding all visits as an array within each patient document. The wide format can improve read performance but complicates updates and wastes space when observations are sparse. The choice between formats involves tradeoffs in storage efficiency, query patterns, and update complexity; these tradeoffs are analogous to the considerations of efficiency vs power in statistical data analysis, or the exploration/exploitation tradeoff in control. See [84] for a comprehensive discussion of database selection in application development.

Recall that the causal methods discussed, such as the SCM and MSM, conceptualize decision-making problems in “wide” format. We will see that classic Reinforcement Learning methods will typically approach these problems in “long” format instead. This mentality permeates through the mathematical and programmatic languages each field uses to describe the problem: R flattens, making code wide and deploying extensive use of SQL commands through the popular package `dplyr`. Causal Inference will also typically use two indexing variables in notation: i for subject and t for time within subject. Occasionally further indexes are added for patient covariates. In contrast, MDP language will just use one index, j , and deploy in Python through “vertical” programming of object classes as discussed above.

The game of Brazilian Jiu Jitsu can also be framed as a Markov Decision Process. Lamas, et al [89] created a formulation where a BJJ match is characterized by the MDP tuple $(\Omega, \mathcal{A}, \Gamma, \rho)$.

The researchers identified 12 main tactical actions:

- Clinch dispute (CDP) - both competitors standing, seeking dominant position
- Pull guard (PGD) - competitor sits down while pulling opponent into guard
- Takedown attempt/success (TKDA/TKD) - projecting opponent to the floor
- Sweep attempt/success (SWPA/SWP) - switching positions from bottom to top
- Guard pass attempt/success (GPSA/GPS) - escaping opponent’s guard control
- Back-take attempt/success (BTKA/BTK) - achieving dominant rear position
- Submission attempt/success (SUBA/SUB) - applying chokes or joint locks

The action space, therefore, corresponds to the set of tactical decisions about which technique to attempt next, with the constraint that certain transitions are impossible due to positional requirements; for example, one cannot escape the opponent's guard unless one is in the opponent's guard. Each state is distinguished by which competitor (self or opponent) performs the action, for a resulting state space of $|\Omega| = 24$.

The researchers defined a reward structure for each state based on *reward-risk balance* (RRB) as in Equation 4.9.

$$\text{RRB}(x, a) = P(\text{eventual submission by actor}) - P(\text{eventual submission by opponent}) \quad (4.9)$$

This reward captures the long-term strategic value of each action, accounting for both offensive potential and defensive vulnerability.

The MDP includes two types of absorbing states:

1. **Submission (SUB):** Match terminates with victory
2. **Reset:** Competitors return to standing position (clinch dispute)

The researchers estimated transition probabilities from observational data of 93 elite-level matches. Analysis revealed that back-takes had the highest RRB (0.55), making them the most advantageous tactical position. Back takes were not, however, the most frequently attempted action: Players attempted submissions most frequently (1.03 per match). Since it is possible to submit the opponent from the back, this result is unsurprising.

The Reinforcement Learning framework is built to solve the problem where one wishes to maximize the long-term rewards by manipulating the action vector. The policy π consists of these manipulations to the action vector. More formally, our goal is to learn an optimal policy π that maximizes our rewards. To quote Sutton and Barto in [164], “Reinforcement Learning methods specify how the agent's policy is changed as a result of its experience”.

The action vector is also sometimes called the “control vector”: Optimal Control is the larger field of theories which are built to solve the maximum returns problem.

The expected long-term rewards are formulated using one of two related conditional expectations: the *value function* $V_\pi(x) = \mathbb{E}_\pi[\text{cumulative return} \mid X = x]$ (Equation 4.10) and the *Q function* $Q_\pi(x, a) = \mathbb{E}_\pi[\text{cumulative return} \mid X = x, A = a]$ (Equation 4.11). The Q function conditions on both state and action; the value function conditions only on state and averages over actions according to the policy. Following Sutton and Barto (with the exception that they use s instead of x for state), “we define the value of taking action a in state $[x]$ under a policy π , denoted $[q_\pi(x, a)]$, as the expected return starting from $[x]$, taking the action a , and thereafter following policy π ”.[164]

The value of a state-action pair is a type of covariate-adjusted potential outcome, with the state vector X containing covariates. One difference lies in the way that future potential outcomes are incorporated into the selection of a treatment for an incoming patient. The value function explicitly seeks not only to maximize the immediate returns, but also the rewards from future actions; present and future rewards are considered in a single equation to be maximized. Conversely, the Causal Inference-based potential outcomes framework would typically split this process into several steps:

1. Find a model which can predict the state mechanics X_{j+1} given action a_j and covariates x_j
2. Use this model to estimate future state transitions for the specific subject until the terminal state is predicted

3. Repeat for all possible actions
4. Estimate some causal effect (e.g. causal risk ratio or CATE)

Causal Inference methods typically also skip the MDP formulation, proceeding straight to prediction of $R_{j+1} = \rho(X_{j+1})|x_j, a_j$ via the modeling methods described in Chapter 2. Some Reinforcement Learning methods might also include the above four steps, but they are oriented towards optimization of the value function *via* an optimal policy. Causal Inference methods, on the other hand, are more oriented towards discovering the fundamental properties of actions than deriving optimal treatment strategies; the “end goal” or teleology is the estimation of the causal effect. Even in Causal Inference, however, there is naturally a latent implication of optimal treatment using the actions with the desirable properties once these properties are known.

4.4 Monte Carlo Methods

A simple technique called Monte Carlo estimation uses the value function to extend the action-value method; Monte Carlo methods estimate the value of a state by averaging the returns observed after visiting that state. For a policy π , the Monte Carlo estimate of the value function after J trials is given in Equation 4.12. The dashed lines in Figure 4.2 are Monte Carlo estimates of each treatment group, computed from their respective observed rewards, which appear as the scattered points: triangles are observed rewards from group 2, whilst circles are observed rewards from group 1. The dotted line shows the exploration probability ϵ .

Definition 4.4 (Value function) *The value function $V_\pi : \Omega \rightarrow \mathbb{R}$ for a policy π is defined as the expected cumulative discounted reward when starting from state x and thereafter following policy π (Equation 4.10).*

$$V_\pi(x) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{j+k+1} \mid X_j = x \right], \quad \gamma \in [0, 1) \quad (4.10)$$

Here, the expectation $\mathbb{E}_\pi$ is taken over the distribution of trajectories induced by selecting actions according to π and transitioning according to Γ .

In Equations 4.10-4.11, γ is a constant called the “discount factor” which prioritizes the more immediate rewards.

Definition 4.5 (Q-function (Action-Value Function)) *The action-value function or Q-function $Q_\pi : \Omega \times \mathcal{A} \rightarrow \mathbb{R}$ for a policy π is defined, per Equation 4.11, as the expected cumulative discounted reward when starting from state x , taking action a , and thereafter following policy π .*

$$Q_\pi(x, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{j+k+1} \mid X_j = x, A_j = a \right], \quad \gamma \in [0, 1) \quad (4.11)$$

The Q-function and value function are related by $V_\pi(x) = \sum_{a \in \mathcal{A}} \pi(a|x) Q_\pi(x, a)$.

The value function, as defined in Equation 4.10, provides a formal definition of the long-term expected return, but it does not immediately suggest how to compute this value in

practice. Monte Carlo methods offer one approach for estimating value functions directly from experience.

Monte Carlo methods come in two primary variants: First-visit and every-visit. First-visit Monte Carlo averages returns only for the first occurrence of a state within an episode, while every-visit Monte Carlo averages returns for all occurrences. First-visit Monte Carlo is unbiased and has been the most widely studied; every-visit Monte Carlo is biased but consistent, and extends more naturally to function approximation [164]. However, modern AI solutions rarely use pure Monte Carlo optimization in practice (we include them here for conceptual foundation). More sophisticated approaches such as Q-learning are typically preferred; we will return to these in future chapters.

For a policy π , the Monte Carlo estimate of the value function averages the observed returns G_j from all visits to each state. Let $G_j = \sum_{k=0}^{T-j-1} \gamma^k R_{j+k+1}$ denote the discounted return observed following the visit to state X_j at step j within an episode of length T . After observing J total state visits across all episodes, the first-visit Monte Carlo estimate is given in Equation 4.12.

$$\hat{V}_\pi(x) = \frac{\sum_{j=1}^J G_j \mathbb{1}(X_j = x \text{ and first visit})}{\sum_{j=1}^J \mathbb{1}(X_j = x \text{ and first visit})} \quad (4.12)$$

The every-visit variant differs from 4.12 in that it removes the “first visit” restriction, averaging returns from all visits to each state instead of just the first one.

Monte Carlo methods embrace an empiricist philosophy by relying solely on complete observations rather than models or bootstrapped estimates. Consequently, Monte Carlo methods are particularly attractive in clinical settings where the priority is to base treatment decisions on directly observed outcomes rather than theoretical predictions.

We can use Monte Carlo estimation to evaluate different BJJ strategies. One strategy might be to always pick the highest-reward action, such as a takedown or submission, regardless of how likely it is to succeed. This would be a very aggressive strategy. Conversely, one might pick the safest option by default, choosing to set up the position perfectly before attempting a submission. If the position is set up properly, then the submission is more likely to work, at the expense of the time and energy investment on setting up the position, which might in itself be unsuccessful. We parameterize the game of BJJ as a Markov Decision Process based on Lamas, et al [89], but with some simplifying adjustments: Instead of 12 actions, we reduce the space to 5, considering only the clinch, guard, guard pass, back take, and submission actions. This simplification makes the code more readable and accessible. Additionally, we only consider the game from a single players perspective instead of modeling both players’ actions, which greatly simplifies the computational approach. Algorithm 2 shows the pseudo-code for this procedure; the complete Python implementation is available in Appendix D.1 and in the online supplement.

Algorithm 2 Monte Carlo Policy Evaluation for BJJ Strategy

Require: Policy π , number of episodes N , discount factor γ
Ensure: Value function estimate $\hat{V}^\pi(x)$

- 1: **Initialize:** Returns(x) $\leftarrow$ empty list for all $x \in \mathcal{X}$
- 2: $\mathcal{X} = \{\text{Clinch}, \text{Guard}, \text{GuardPass}, \text{BackTake}, \text{Submission}, \text{Reset}\}$
- 3: **for** episode $i = 1$ to N **do**
- 4: Generate episode $\mathcal{E} = \{(x_0, a_0, r_0), \dots, (x_T, a_T, r_T)\}$ following π
- 5: Initialize return: $G \leftarrow 0$
- 6: Initialize visited states: $\mathcal{V} \leftarrow \emptyset$
- 7: **for** $t = T$ down to 0 **do** $\triangleright$ Backward pass through episode
- 8: $G \leftarrow \gamma \cdot G + r_t$ $\triangleright$ Accumulate discounted return
- 9: **if** $x_t \notin \mathcal{V}$ **then** $\triangleright$ First-visit check
- 10: Append G to Returns(x_t)
- 11: $\mathcal{V} \leftarrow \mathcal{V} \cup \{x_t\}$
- 12: **end if**
- 13: **end for**
- 14: **end for**
- 15: **Compute value estimates:**
- 16: **for** each state $x \in \mathcal{X}$ **do**
- 17: **if** Returns(x) is non-empty **then**
- 18: $\hat{V}^\pi(x) \leftarrow \frac{1}{|\text{Returns}(x)|} \sum_{g \in \text{Returns}(x)} g$
- 19: **else**
- 20: $\hat{V}^\pi(x) \leftarrow 0$
- 21: **end if**
- 22: **end for**
- 23: **return** $\hat{V}^\pi$

The result is as follows:

```
Evaluating BJJ Policies using Monte Carlo Methods...
=====
Value Function for Aggressive Policy (pi_aggressive):
V^pi(Clinch      ) =  4.41
V^pi(Guard       ) =  4.99
V^pi(GuardPass   ) =  3.97
V^pi(BackTake    ) =  0.00

Value Function for Positional Policy (pi_positional):
V^pi(Clinch      ) =  3.42
V^pi(Guard       ) =  3.70
V^pi(GuardPass   ) =  6.19
V^pi(BackTake    ) =  6.49

Policy Comparison (Aggressive - Positional):
=====
Clinch      : +0.99 (Aggressive better)
Guard       : +1.29 (Aggressive better)
GuardPass   : -2.22 (Positional better)
BackTake    : -6.49 (Positional better)
```

Overall Expected Return:

Aggressive Policy: 4.41

Positional Policy: 3.42

In this extremely simplified environment, aggressive plays seem to work best from clinch and guard positions, while moving strategically seems to work better from open guard passing and back attempts.

4.5 Importance Sampling

While basic Monte Carlo methods require that we follow policy π to generate our trials, many times this requirement is infeasible: We cannot justify trying new cancer treatments randomly, for example, as the effects of an incorrect treatment selection can be a matter of life and death. Fortunately, statisticians have devised a workaround to this issue called *importance sampling*, which allows us to estimate the value of policy π using trials generated by a different policy b .

Importance sampling adjusts for the discrepancy between policies by weighting returns by the ratio of their probabilities under the target and behavior policies. For a trial sequence of length T , the importance sampling ratio attached to the return G_j is $\omega_j = \prod_{j'=j}^{T-1} \frac{\pi(a_{j'}|X_{j'})}{\pi_b(a_{j'}|X_{j'})}$ (note that the product runs over the actions taken from step j through the end of the sequence, since these are the actions that generate the return following step j). The ordinary importance sampling estimator for the value function after J episodes is then defined as in Equation 4.13.

$$\hat{V}_\pi(X) = \frac{\sum_{j=1}^J \omega_j G_j \mathbb{1}(X_j = X)}{\sum_{j=1}^J \mathbb{1}(X_j = X)} \quad (4.13)$$

The estimator $\hat{V}_\pi(X)$ is unbiased but often exhibits high variance, especially for long trial sequences. An alternative is weighted importance sampling, as indicated in Equation 4.14.

$$\hat{V}_\pi(X) = \frac{\sum_{j=1}^J \omega_j G_j \mathbb{1}(X_j = X)}{\sum_{j=1}^J \omega_j \mathbb{1}(X_j = X)} \quad (4.14)$$

Importance sampling in Reinforcement Learning is a generalization of Inverse Probability of Treatment Weighting (IPTW) discussed in Chapter 2. Both techniques correct for selection bias by weighting outcomes inversely proportional to their selection probability [62].

Proposition 1 (IPTW as Single-Step Importance Sampling) *Consider a single-step MDP (i.e., $j = 1$, one decision point) where the state X represents patient covariates, the action $A \in \{0, 1\}$ is binary treatment assignment, and the reward R is the observed outcome. Let $\pi_b(a|x) = \pi(A = a|X = x)$ be the behavior policy (the observed treatment-assignment mechanism, i.e., the propensity score), and let $\pi_1(a|x) = \mathbb{1}(a = 1)$ be a target policy that assigns everyone to treatment. Then the importance-sampling estimator for the value of π_1*

reduces to the treated-arm component of the IPTW estimator (see Equation 4.15).

$$\hat{V}_{\pi_1} = \frac{1}{n} \sum_{j=1}^n \frac{\pi_1(A_j|X_j)}{\pi_b(A_j|X_j)} R_j = \frac{1}{n} \sum_{j=1}^n \frac{A_j R_j}{\pi(1|X_j)}. \quad (4.15)$$

Similarly, setting $\pi_0(a|x) = \mathbb{1}(a = 0)$ and taking the difference $\hat{V}_{\pi_1} - \hat{V}_{\pi_0}$ recovers the full IPTW estimator for the ATE (Equation 3.5 in Chapter 3).

Proof. For the target policy $\pi_1(a|x) = \mathbb{1}(a = 1)$, the importance sampling ratio for subject j is expressed in Equation 4.16.

$$\omega_j = \frac{\pi_1(A_j|X_j)}{\pi_b(A_j|X_j)} \quad (4.16)$$

When $A_j = 1$ (treated), $\omega_j = 1/\pi_b(1|X_j) = 1/\pi(1|X_j)$. When $A_j = 0$ (untreated), then $\omega_j = 0/\pi_b(0|X_j) = 0$ (untreated subjects contribute nothing). Substituting into the ordinary IS estimator (Equation 4.13 with a single time step, so $G_j = R_j$, and averaging over all subjects rather than conditioning on a particular state x) gives us Equation 4.17.

$$\hat{V}_{\pi_1} = \frac{1}{n} \sum_{j=1}^n \omega_j R_j = \frac{1}{n} \sum_{j=1}^n \frac{A_j R_j}{\pi(1|X_j)} \quad (4.17)$$

The same argument with $\pi_0(a|x) = \mathbb{1}(a = 0)$ gives $\hat{V}_{\pi_0} = \frac{1}{n} \sum_{j=1}^n \frac{(1-A_j)R_j}{1-\pi(1|X_j)}$. Then, their difference is $\hat{V}_{\pi_1} - \hat{V}_{\pi_0} = \frac{1}{n} \sum_{j=1}^n \frac{A_j R_j}{\pi(1|X_j)} - \frac{1}{n} \sum_{j=1}^n \frac{(1-A_j)R_j}{1-\pi(1|X_j)}$, which is the IPTW ATE estimator. $\square$

Proposition 1 tells us that the relationship between IPTW and importance sampling can be made very precise. The IPTW estimator *is* importance sampling, restricted to a single time step with binary actions. Conversely, the multi-step IS ratio $\omega_j = \prod_{j'=j}^{T-1} \frac{\pi(a_{j'}|X_{j'})}{\pi_b(a_{j'}|X_{j'})}$ generalizes IPTW to sequential decisions; this is the premise of the marginal structural modelling strategy of Section 3.4. The mathematical relationship between importance sampling in Reinforcement Learning and inverse probability weighting in Causal Inference has been established in the contextual bandit setting in [44].

Importance sampling might be used to extend the BJJ Value Estimation example in the case where we wish to evaluate the efficacy of behavior policies not taken: Suppose I played a match mostly strategically, strongly preferring to establish position before attempting a submission. I may wish to know, would I have won that match if I had played aggressively? One may answer this question using the importance sampling method described above.

4.6 Further Reading

For a comprehensive treatment of Reinforcement Learning fundamentals, the classic text by Sutton and Barto [164] remains the definitive reference. Their work provides extensive coverage of tabular methods, action-value estimation, and Monte Carlo approaches that complement the material presented in this chapter.

Bayesian approaches to adaptive clinical trial design have been explored in detail in [16] and [166]. For a more technical treatment of the statistical properties of adaptive designs, see [68].

The mathematical foundations of multi-armed bandit problems, which underlie many adaptive clinical trial designs, are thoroughly covered in [28], which provides rigorous analysis of exploration-exploitation trade-offs and regret bounds.

For readers interested in the philosophical underpinnings of Reinforcement Learning, Skinner's "Science and Human Behavior" [158] remains a seminal work on behaviorist approaches to learning. The contrast between idealist and utilitarian philosophies in Reinforcement Learning versus Causal Inference is further explored by Russell and Norvig [141] in their discussion of rational agents. A compelling criticism of practical behaviorism can be found in [85].

Monte Carlo methods and their applications to Reinforcement Learning are covered extensively in Liu [99]. For a deeper understanding of importance sampling techniques, see the comprehensive treatment by Robert and Casella [129]. Additional perspectives on Monte Carlo methods in the context of Bayesian inference can be found in Gelman et al. [51].

Recent advances in applying Reinforcement Learning to clinical decision-making, building on the foundations presented in this chapter, can be found in Chakraborty and Murphy [34], whose work on dynamic treatment regimes bridges the gap between traditional clinical trial methodology and modern Reinforcement Learning approaches. The previously-mentioned cancer-specific book on adaptive clinical trials [88] is also an excellent reference.